

1-Billion-Token Context Window. It's Absolutely Unreal.

Sharath

Preprint · Version 1.0.0 · 8 August 2026

<https://doi.org/10.5281/zenodo.21852829>

Licensed under CC BY 4.0

Abstract

I built a system that exposes a 1-billion-token context window through a standard OpenAI-compatible chat API, powered by a 4B parameter model on a single machine. The method: accept up to 4 GB of text, index it in RAM on the fly, and let a small controller model search through it before answering. The client has no idea retrieval is happening. It just looks like a model with an enormous context window.

1. What If You Just Lied?

Everyone's racing to extend context windows. Gemini does 1M tokens. Claude does 200K. GPT-4 does 128K. Each jump requires serious architectural work---sparse attention, ring attention, hierarchical memory---and costs a fortune in compute to train and serve.

I had a different idea: what if I just said "1 billion tokens" and then made it work without actually extending attention?

Here's the insight. When someone shoves a giant document into a model and asks a question, they almost never need the model to attend to every token simultaneously. They need it to find the right paragraph and reason about it. That's what humans do. You don't re-read an entire book every time someone asks you a question about it---you flip to the relevant page.

So that's what biggie-kun does. It accepts up to 4 GB of text in one HTTP request. It builds a search index over that text in memory. It uses a small local model (Qwen 3.5 4B via Ollama) to search, read relevant chunks, and answer. And it does all of this behind the standard `/v1/chat/completions` endpoint that every OpenAI client already speaks.

The client sends messages. The client gets an answer. It never knows retrieval happened.

2. Everyone Else Is Doing It The Hard Way

Extending Attention

The serious approaches to long context all modify the transformer itself:

- **RoPE scaling** (Chen et al., 2023): Interpolate position embeddings to handle longer sequences without retraining from scratch.
- **Ring Attention** (Liu et al., 2023): Distribute attention across devices in a ring so no single machine needs to hold the full KV cache.
- **Infini-attention** (Munkhdalai et al., 2024): Combine compressive memory with local attention windows.
- **Mamba** (Gu & Dao, 2023): Replace attention entirely with selective state spaces that scale linearly.

All of these require training at the target length and scale compute with sequence length. They're solving a genuinely hard problem. I'm just avoiding it.

Traditional RAG

RAG (Lewis et al., 2020) is the established workaround: pre-index documents, retrieve relevant chunks at query time, stuff them into the prompt. But traditional RAG:

1. Needs a separate ingestion step (upload, embed, store).
2. Needs a vector database.
3. Needs an embedding model.
4. Exposes a different API than a normal chat model.
5. Requires the client to know it's doing RAG.

What I Did

I took RAG and hid it completely inside a model endpoint. No upload step. No vector store. No embeddings. No client awareness. You send messages, you get an answer. The "context window" is just however much text you put in the request body. From the outside, it's indistinguishable from a model that natively handles a billion tokens.

3. The World's Most Disposable Search Engine

The core of the system is an ephemeral inverted index that gets built from scratch on every request and thrown away when the response is done.

Chopping Text Into Blocks

The input gets split into overlapping ~16 KB blocks. Boundaries snap to the nearest newline so you don't cut sentences in half. Adjacent blocks overlap by 512 bytes so nothing falls between cracks.

A 4 GB corpus produces roughly 250,000 blocks.

The Inverted Index

Each block's unique terms get recorded in a map from word to list of block IDs containing that word. Two hard caps keep memory bounded:

- **Max vocabulary: 250,000 terms.** After this, new terms are ignored.
- **Max postings per term: 4,096 blocks.** If a word appears in more than 4,096 blocks, it's too common to be a useful search key---drop it entirely.

That second cap is doing a lot of work. Words like "the" or "function" appear everywhere and are useless for retrieval. By capping posting lists, they get automatically discarded without needing a hand-tuned stopwords list.

Search: Two Phases

Phase 1---IDF scoring. Query terms get looked up in the index. Each block in a term's posting list gets a score weighted by how rare that term is:

```
score += log(1 + total_blocks / blocks_containing_term)
```

Rare terms score high. Common terms score low. Standard information retrieval intuition.

Phase 2---Exact match bonus. Top candidates get scanned for verbatim substring matches against the full query. Matches get a large bonus (+20). This heavily favors exact phrase hits, which is what you want when the user is looking for a specific name, ID, or quote.

Top 8-12 results come back with ~1,280-character snippets centered on the match.

Reading

When the agent wants the full text of a block, it gets it plus optionally one neighbor on each side. A byte budget (400 KB default) caps total evidence per request. No matter how big the corpus is, the model never sees more than this bounded window.

Why This Works At Scale

After the one-time linear pass to build the index, every subsequent operation is bounded by constants. Search doesn't get slower as the corpus grows---it's always bounded by posting list length. The 4 GB corpus and the 40 KB corpus use the same retrieval path with the same worst-case cost.

4. The Tiny Model That Runs The Show

The retrieval loop is controlled by a small LLM that gets three actions and a turn budget.

Three Buttons

Action	What happens
<code>search</code>	Send queries to the index, get ranked snippets back
<code>read</code>	Get full text of search results
<code>answer</code>	"I have enough, let me answer"

Actions are enforced with JSON Schema constraints. The model literally cannot output an invalid action because generation is constrained to match the schema.

No Spinning

A simple state machine prevents degenerate behavior:

- From start: search or answer
- After search: read or answer (no double-searching)
- After read: answer or search again (with new terms you discovered)

This forces progress. The agent can't just search repeatedly without reading, or read without ever searching.

Hard Walls

Every turn, the agent sees its budget:

```
Turn 3/12; scanned 48000/400000 bytes.
```

12 turns max. 400 KB evidence budget. These are non-negotiable. The model knows when it's running low and should wrap up.

Forced Termination

If the agent blows through all 12 turns without saying "answer," the system forces an answer generation pass using whatever evidence was collected. Every request produces a response no matter what the controller does.

The Typical Run

Most questions resolve in 3-5 turns: search once, read 2-4 blocks, answer. The agent figures out good search terms from the question, finds the relevant blocks, reads them, and answers. Multi-hop questions (where the first result gives you a name to search for next) might take 6-8 turns.

5. Does It Actually Work?

Where It's Great

- **Needle in a haystack.** Finding a specific fact buried in gigabytes of text. If the answer contains a rare term, the index finds it instantly regardless of corpus size.
- **Named entity lookups.** "What did Sarah say about the deadline?" Person names are rare terms. Index nails these.
- **Multi-hop questions.** "Who manages the person that filed ticket #4521?" The agent searches for the ticket, finds the filer's name, searches for that name, finds their manager.
- **Drop-in compatibility.** Anything that talks to the OpenAI API works without changes.

Where It Falls Apart

- **"Summarize the whole thing."** You can't summarize 4 GB from 400 KB of samples. This requires global attention over all content. We literally don't have that.
- **Semantic-only queries.** If your question shares no words with the answer, the lexical index can't find it. No embeddings means no fuzzy semantic matching.
- **Counting.** "How many errors are in this log?" requires exhaustive scanning. We sample.
- **Vibes.** "What's the tone of this document?" requires reading all of it.

The Billion-Token Test

The repo includes a test that constructs a 4,000,000,000-byte corpus (1 billion estimated tokens), plants a unique synthetic record near the end, and verifies correct retrieval:

```
BIGGIE_REAL_TEST_BYTES=4000000000 \  
go test -run TestRealHundredMillionTokenIndex -count=1 -v
```

It works. The index finds a unique needle in 4 GB of hay. Default CI runs over 400 MB and finishes in seconds on commodity hardware.

6. OK But Is This Actually Cheating?

Yes

I don't extend attention. I don't train on long sequences. I don't compute over the full context. I hide a retrieval system behind a model API and put "1 billion tokens" on the label because technically you can send that many tokens and get a correct answer back.

But Does It Matter?

For the stuff people actually use large contexts for---document Q&A, code search, log analysis, research---you almost never need global attention. You need the system to find the right passage and think about it. By that standard, this is a legitimate billion-token context window for the majority of real workloads.

"Context Window" Is An Interface Contract

Think about what a context window actually promises. It says: "you can include up to N tokens in your request and I will use them to answer your question."

It does not say "I will attend to every token with quadratic compute." It does not say "I will read every word."

Native attention models fulfill this contract by processing everything. I fulfill it by indexing everything and processing what's relevant. Same interface. Different implementation. Both valid.

The Cost Difference Is Absurd

Approach	Hardware	Cost
Native 1B-token attention (hypothetical)	Multi-GPU cluster, custom training	\$\$\$\$\$
biggie-kun	One machine, 32 GB RAM, any GPU	Electricity

Not a marginal difference. Orders of magnitude.

7. TL;DR

I made a 1-billion-token context window by accepting 4 GB of text, building a disposable search index in RAM, and letting a 4B parameter model poke around in it before answering. The client never knows. It just looks like a very patient model with an impossibly large context window.

The honest framing: agentic retrieval hidden behind a model endpoint.

The marketing framing: 1 billion token context window.

Both are true. Depends which layer you look at.

Sometimes the best way to solve a hard problem is to not solve it and just make it look like you did.

References

1. Chen, S., Wong, S., Chen, L., & Tian, Y. (2023). Extending context window of large language models via positional interpolation. *arXiv:2306.15595*.
2. Liu, H., Zaharia, M., & Abbeel, P. (2023). Ring Attention with blockwise transformers for near-infinite context. *arXiv:2310.01889*.
3. Munkhdalai, T., Faruqui, M., & Gopal, S. (2024). Leave no context behind: Efficient infinite context transformers with Infini-attention. *arXiv:2404.07143*.
4. Gu, A., & Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*.
5. Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.